

SENDIPAY

Plateforme de Comparaison et de Suivi des Transferts d'Argent pour la
Diaspora Africaine

Rapport de Projet — Génie Logiciel Cloud

Yaovi Urbain

Juin 2026

1. Résumé Exécutif

SendiPay est une application web cloud-native conçue pour répondre à un problème concret vécu par des millions de personnes : le coût élevé des transferts d'argent vers l'Afrique subsaharienne. Cette plateforme permet aux utilisateurs de comparer en temps réel les frais pratiqués par les principaux opérateurs de transfert (Wave, Orange Money, WorldRemit, Western Union), de suivre leur historique d'envois et de configurer des alertes de taux de change.

L'application est déployée en production sur des plateformes cloud publiques : le frontend sur Vercel et le backend sur Render, avec une base de données MongoDB Atlas. Elle est accessible via un lien public et démontre une architecture moderne, évolutive et sécurisée, conformément aux exigences du génie logiciel cloud.

2. Identification du Problème

2.1 Contexte et Justification

Les transferts d'argent internationaux représentent une source vitale de revenus pour de nombreux pays d'Afrique subsaharienne. Selon la Banque mondiale, les flux de remises vers l'Afrique subsaharienne dépassent 50 milliards de dollars américains par an, surpassant l'aide publique au développement dans de nombreux pays. Le Sénégal, par exemple, reçoit des remises représentant plus de 10 % de son PIB.

Cependant, les frais de transfert restent excessivement élevés. Le coût moyen mondial d'un transfert de 200 dollars est de l'ordre de 6 à 9 %, bien au-dessus de l'objectif de 3 % fixé par les Objectifs de Développement Durable (ODD) des Nations Unies (cible 10.c). Cette situation représente un manque à gagner considérable pour les familles bénéficiaires.

2.2 Problèmes Identifiés

- Manque de transparence sur les frais réels pratiqués par chaque opérateur
- Absence d'outil centralisé de comparaison des offres en temps réel
- Difficulté pour les utilisateurs de suivre leur budget de transfert mensuel
- Aucune solution d'alerte pour profiter des taux de change favorables
- Dispersion des informations entre plusieurs applications et sites web

2.3 Solution Proposée

SendiPay répond à ces problèmes en centralisant toutes ces fonctionnalités dans une interface web unique, intuitive et accessible depuis tout navigateur. La solution cible

principalement la diaspora sénégalaise et ouest-africaine en Europe, mais est conçue pour être étendue à d'autres corridors de transfert.

3. Architecture du Système

3.1 Vue d'Ensemble

SendiPay adopte une architecture three-tier classique adaptée aux contraintes du cloud moderne : une couche présentation (frontend), une couche métier (backend API REST) et une couche données (base de données). Cette séparation des responsabilités garantit la maintenabilité, la testabilité et l'évolutivité du système.

Couche	Technologie	Hébergement	Rôle
Frontend	React 18 + Vite + Tailwind CSS	Vercel (Edge CDN)	Interface utilisateur
Backend	Node.js + Express 4	Render.com	API REST + logique métier
Base de données	MongoDB Atlas	AWS (cloud)	Persistance des données
Auth	JWT + bcryptjs	Embarqué backend	Sécurité et sessions
API externe	ExchangeRate API	Tiers	Taux de change temps réel

3.2 Architecture du Backend

Le backend suit le pattern MVC (Model-View-Controller) adapté aux APIs REST, organisé en couches distinctes :

- Models : Schémas Mongoose définissant la structure des documents MongoDB (User, Transfer, Alert)
- Controllers : Logique métier isolée par domaine fonctionnel (auth, transferts, comparateur, alertes)
- Routes : Définition des endpoints HTTP et application des middlewares
- Middleware : Authentification JWT, rate limiting, validation des entrées

3.3 Modèle de Données MongoDB

Le choix de MongoDB comme base de données repose sur la nature flexible des données manipulées. Les réponses des différents opérateurs de transfert ont des structures variables, ce que le modèle document de MongoDB gère naturellement sans migration de schéma. Trois collections structurent le système :

Collection	Champs principaux	Relations
users	_id, name, email, passwordHash, country, currency, createdAt	Référencée par transferts et alerts
transfers	_id, userId, amount, currency, provider, fees, exchangeRate, amountReceived, status	userId → users._id
alerts	_id, userId, currency, targetRate, active, triggeredAt	userId → users._id

4. Choix Technologiques et Justifications

4.1 Frontend : React + Vite + Tailwind CSS

React 18 a été choisi pour sa maturité, son écosystème riche et sa courbe d'apprentissage adaptée. L'architecture par composants facilite la réutilisation et la maintenance du code. Vite remplace Create React App pour ses performances de développement supérieures (démarrage instantané, HMR rapide). Tailwind CSS permet un design cohérent et responsive sans écrire de CSS personnalisé, réduisant considérablement le temps de développement.

La bibliothèque Recharts, basée sur D3.js, permet la visualisation des comparaisons de frais sous forme de graphiques en barres interactifs. React Router v6 gère la navigation côté client (SPA) avec des routes protégées selon l'état d'authentification.

4.2 Backend : Node.js + Express

Node.js offre d'excellentes performances pour les applications I/O-intensive grâce à son modèle événementiel non-bloquant. Express.js, framework minimaliste et flexible, permet une structuration claire des routes et middlewares. La sécurité est assurée par plusieurs niveaux : hachage des mots de passe avec bcryptjs (salt rounds: 12), authentification stateless via JWT, et rate limiting à 100 requêtes par 15 minutes par IP.

4.3 Base de Données : MongoDB Atlas

MongoDB Atlas offre un tier gratuit (M0) avec 512 MB de stockage, largement suffisant pour un MVP. La plateforme gère automatiquement les sauvegardes, la réplication (Replica Set à 3 nœuds) et la mise à l'échelle. Mongoose comme ODM apporte la validation des données et une API fluide pour interagir avec MongoDB en Node.js.

4.4 Déploiement Cloud

Vercel et Render ont été choisis pour leur simplicité de déploiement (intégration GitHub native), leur tier gratuit et leur support natif des applications Node.js et React. Le déploiement

continu (CI/CD) est automatisé : chaque push sur la branche main déclenche un nouveau déploiement.

5. Développement et Implémentation

5.1 Principes de Codage

Le développement de SendiPay a suivi les principes du Clean Code et de la conception modulaire. Chaque fichier a une responsabilité unique (Single Responsibility Principle). Les contrôleurs sont isolés des routes, facilitant les tests unitaires. La gestion des erreurs est centralisée et cohérente à travers toute l'API.

5.2 Fonctionnalités Implémentées

Fonctionnalité	Description	Endpoint
Inscription / Connexion	Création de compte et authentification sécurisée	POST /api/auth/register, /login
Comparateur de frais	Comparaison temps réel de 4 opérateurs avec graphique	GET /api/comparator
Historique des envois	Ajout, consultation et suppression des transferts	GET/POST/DELETE /api/transfers
Statistiques	Total envoyé, frais cumulés, nombre de transferts	Inclus dans GET /api/transfers
Alertes de taux	Configuration de seuils de taux de change	GET/POST/DELETE /api/alerts
Route de santé	Vérification du statut de l'API	GET /api/health

5.3 Sécurité

Plusieurs mesures de sécurité ont été implémentées dans SendiPay. Les mots de passe sont hachés avec bcryptjs avant stockage, rendant toute compromission de la base de données inutilisable. L'authentification JWT stateless évite les sessions côté serveur. La politique CORS restreint les origines autorisées. Le rate limiting protège contre les attaques par force brute. La validation des entrées avec express-validator prévient les injections.

6. Déploiement Cloud

6.1 Infrastructure de Production

SendiPay est déployé sur une infrastructure entièrement cloud, accessible publiquement. L'architecture de déploiement sépare clairement les responsabilités entre les différentes plateformes :

Composant	Plateforme	Caractéristiques
Frontend	Vercel	CDN mondial, HTTPS automatique, déploiement en 30s
Backend API	Render.com	Node.js natif, variables d'environnement sécurisées, logs en temps réel
Base de données	MongoDB Atlas (AWS)	Replica Set 3 nœuds, sauvegardes automatiques, 512 MB gratuit

6.2 Pipeline CI/CD

Le pipeline d'intégration et de déploiement continu est entièrement automatisé via l'intégration GitHub de Vercel et Render. Chaque commit sur la branche main déclenche simultanément le build et le déploiement du frontend sur Vercel et le redémarrage du service backend sur Render. Ce processus garantit que la version en production reflète toujours le dernier état du code source.

6.3 Variables d'Environnement et Sécurité

Toutes les informations sensibles (URI MongoDB, secret JWT, clés API) sont stockées comme variables d'environnement sur les plateformes cloud, jamais dans le code source. Le fichier .gitignore exclut le fichier .env local du dépôt GitHub. Cette approche suit les bonnes pratiques de sécurité du développement cloud-native (Twelve-Factor App).

7. Résultats et Validation

7.1 Tests Fonctionnels

L'application a été validée à travers une série de tests fonctionnels couvrant l'ensemble des parcours utilisateur :

- Inscription d'un nouvel utilisateur avec validation des champs
- Connexion et persistance de session via JWT stocké en localStorage
- Comparaison des frais pour différents montants et devises (EUR, USD, GBP)
- Ajout et consultation de l'historique des transferts
- Calcul correct des statistiques (total envoyé, total frais)
- Création et suppression d'alertes de taux de change

- Protection des routes privées (redirection vers login si non authentifié)

7.2 Performances et Accessibilité

L'application est accessible via HTTPS sur les deux domaines. Le frontend bénéficie du CDN mondial de Vercel pour des temps de chargement optimisés. Le backend sur Render utilise le tier gratuit avec une latence acceptable pour un prototype (cold start de 30-50 secondes après inactivité, puis réponses en moins de 200 ms). Le design responsive assure une expérience utilisateur cohérente sur desktop et mobile.

8. Discussion et Perspectives

8.1 Apports du Projet

Ce projet démontre la capacité à concevoir et déployer une application web complète en suivant les standards professionnels du génie logiciel moderne. L'approche itérative a permis d'identifier et de résoudre des problèmes concrets de déploiement (CORS, routage SPA, variables d'environnement), renforçant la compréhension des défis réels du cloud.

8.2 Limitations Actuelles

- Les données des opérateurs sont actuellement statiques (frais approximatifs) — une intégration aux APIs officielles est nécessaire
- Le tier gratuit de Render implique un cold start après inactivité
- L'envoi d'emails pour les alertes nécessite une configuration SMTP complète
- Absence de tests automatisés (unité, intégration) dans la version actuelle

8.3 Évolutions Futures

- Intégration aux APIs officielles de Wave, Orange Money et WorldRemit
- Implémentation des tests unitaires et d'intégration avec Jest et Supertest
- Ajout d'une Progressive Web App (PWA) pour l'utilisation hors-ligne
- Tableau de bord analytique avec graphiques d'évolution des taux
- Système de notifications push pour les alertes de taux
- Support multi-devises élargi (GNF, XAF, MAD, NGN)

9. Conclusion

SendiPay illustre comment les technologies web modernes et les plateformes cloud peuvent être mobilisées pour résoudre des problèmes d'inclusion financière concrets. En combinant

une architecture three-tier rigoureuse, un déploiement cloud automatisé et une interface utilisateur intuitive, ce projet démontre une maîtrise des compétences essentielles du génie logiciel moderne.

La démarche adoptée — de l'identification du problème jusqu'au déploiement en production — reflète une approche professionnelle et autonome de la résolution de problèmes informatiques complexes. Le projet est évolutif, sécurisé et prêt à accueillir de nouvelles fonctionnalités dans le cadre d'une feuille de route produit.

Références

- Banque Mondiale (2024). Remittance Prices Worldwide. worldbank.org/remittanceprices
- Nations Unies. Objectifs de Développement Durable, Cible 10.c — Réduire les coûts de transfert de fonds
- MongoDB Inc. (2024). MongoDB Atlas Documentation. docs.atlas.mongodb.com
- Vercel Inc. (2024). Vercel Documentation. vercel.com/docs
- Render Inc. (2024). Render Documentation. render.com/docs
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. (REST)
- Wiggins, A. (2017). The Twelve-Factor App. 12factor.net
- OWASP Foundation (2024). OWASP Top Ten Web Application Security Risks